of the methods you used to process the data. In some cases, like the Sankey diagram here or the force-directed network visualization you'll see in the next chapter, the algorithm used to sort and arrange the graphical elements is front and center. After you have a layout that displays properly, you can play with the settings and update the elements like you've done with the Sankey diagram to better understand how the algorithm works visually.

First we need to add an `onclick` function to make the chart interactive, as shown in listing 5.8. We'll attach this function to the `<svg>` element itself, but you could just as easily add a button like we did in chapter 3.

The `moreLayouts()` function does two things. It updates the `sankey.layout()` property by incrementing a variable and setting it to the new value of that variable. It also selects the graphical elements that make up your chart (the `<g>` and `<path>` elements) and redraws them with the updated settings. By using `transition()` and `delay()`, you'll see the chart dynamically adjust.

Listing 5.8 Visual layout function for the Sankey diagram

```
var numLayouts = 1;
d3.select("svg").on("click", moreLayouts);
sankey.layout(numLayouts);

function moreLayouts() {
   numLayouts += 20;
   sankey.layout(numLayouts);

   d3.selectAll(".link")
   .transition()
   .duration(500)
   .attr("d", sankey.link())

   d3.selectAll(".node")
   .transition()
   .duration(500)
   .attr("transform", function(d) {
           return "translate(" + d.x + "," + d.y + ")";
   });
}
```

Initializes the sankey with only a single layout pass

We choose 20 passes because it shows some change without requiring us to click too much.

Because the layout updates the dataset, we just have to call the drawing functions again and they automatically update.



Figure 5.26 The Sankey layout algorithm attempts to optimize the positioning of nodes to reduce overlap. The chart reflects the position of nodes after (from left to right) 1 pass, 20 passes, 40 passes, and 200 passes.